Rapport projet final Python

Moteur d'analyse de logs

ENIZAN Hugo - TORTEL Sami - AHMED MOHAMED ALI Yassin

Introduction..... 3
1. Vue d'ensemble de la chaîne..... 4
2. Lecture en flux avec yield et yield from..... 5
3. Parsing et validation d'une ligne..... 5
4. Génération des données de test et mode --details..... 6
5. Nettoyage et normalisation avec filter, map et lambda..... 7
6. Analyse de structure avec match-case..... 8
7. Moteur de règles avec *args et **kwargs..... 9
8. Audit des portées avec globals() et locals()..... 10
9. Rôle du module d'analyse..... 10
10. Architecture du module d'analyse..... 11
11. Notions Python de l'analyse..... 11
12. Détection de la force brute..... 12
13. Démonstration sur le jeu de référence..... 13
14. Vérifications et mémoire..... 14
15. Limites et améliorations possibles..... 15
Conclusion..... 16

Introduction

Ce projet réalise un moteur en ligne de commande qui lit des journaux de logs volumineux, les nettoie, applique des règles de sécurité et produit un bilan des attaques, sans jamais charger les fichiers en mémoire. Sur un fichier de 10 Go, la chaîne complète traite 56 millions d'événements avec moins de 20 Mo de RAM.

Contexte. Un log est la trace d'un événement sur un système : une connexion réussie ou échouée, une requête, une erreur de service. Ces journaux atteignent vite plusieurs gigaoctets, trop pour être chargés d'un bloc, alors qu'ils contiennent les traces des attaques qu'on cherche à repérer.

Objectif. L'énoncé demande de « concevoir et développer une plateforme complète en ligne de commande (CLI) d'ingestion, de filtrage, de transformation et d'analyse de données système/réseau complexes issues de journaux de logs volumineux ». Concrètement, le programme doit :

1. **lire** des fichiers de logs, un ou plusieurs, ligne par ligne ;
2. **filtrer et transformer** les événements : écarter les lignes invalides, normaliser les valeurs ;
3. **appliquer des règles de sécurité** paramétrables, comme une blacklist d'IP ;
4. **analyser** le flux : compter les événements, repérer les IP suspectes et détecter les attaques par force brute ;
5. **produire un bilan** lisible à la fin du traitement.

Le tout doit traiter « un volume très important de données en mémoire de manière optimisée sans charger l'intégralité des fichiers en RAM », avec une architecture modulaire, un environnement isolé et un code structuré.

Contraintes imposées. Le projet, mené en trinôme en Python 3.10 ou plus, doit respecter :

- un environnement virtuel isolé, reproductible grâce à un `requirements.txt` ;
- un code organisé en package Python, piloté par un point d'entrée unique, `main.py` ;
- quatre modules techniques : l'ingestion par générateurs (`yield`, `yield from`), un pipeline fonctionnel (`filter`, `map`, `lambda`, `match-case`), l'agrégation (`set`, `frozenset`, `defaultdict`, boucles avec `break`, `continue` et `else`) et un moteur de règles avec introspection (`*args`, `**kwargs`, `globals()`, `locals()`) ;
- un code de qualité : PEP 8, typage et gestion des exceptions.

Choix de l'équipe.

- Les logs sont au format JSON Lines : un objet JSON par ligne, avec six champs obligatoires (`timestamp`, `level`, `ip`, `user_id`, `connection`, `payload`) fixés dans un contrat commun.
- Chaque étape reçoit et renvoie un itérateur : un événement traverse toute la chaîne avant que la ligne suivante soit lue.
- Seule la bibliothèque standard de Python est utilisée : `requirements.txt` est vide, et le projet s'installe sans dépendance.

- Un jeu de référence de 10 événements (`data/demo.jsonl`) fixe les résultats attendus, et un générateur produit des fichiers jusqu'à 10 Go pour les essais de volume.
- Le comportement est vérifié par 75 tests `unittest`.

Organisation du rapport. Les sections suivent le trajet d'un événement dans la chaîne :

Partie de l'énoncé	Sections
Ingestion et générateurs	2. Lecture · 3. Parsing · 4. Données de test
Pipeline fonctionnel	5. Nettoyage · 6. match-case
Moteur de règles et introspection	7. Règles · 8. Audit
Agrégation et détection	9 à 12. Analyse et force brute
Résultats	13. Démonstration · 14. Vérifications · 15. Limites

1. Vue d'ensemble de la chaîne

Chaque événement traverse toute la chaîne avant que la ligne suivante soit lue : aucune étape ne garde le fichier ni la liste des événements en mémoire.

```
Lecture   : fichiers -> lignes -> dictionnaires
Nettoyage : lignes vides et invalides écartées -> normalisation
Règles    : alertes ajoutées à chaque événement (champ alerts)
Analyse   : statistiques, ensembles d'IP, force brute -> bilan
```

Étape	Fichiers	Ce qu'elle renvoie
Lecture et parsing	`logengine/ingestion.py`, `logengine/parsing.py`	des lignes une par une, puis un dictionnaire typé ou `None` par ligne
Nettoyage et normalisation	`logengine/pipeline.py`	un itérateur d'événements propres
Règles de sécurité	`logengine/rules.py`	les mêmes événements, avec leur liste `alerts`
Audit des portées	`logengine/audit.py`	un fichier JSON des noms et types de variables
Analyse et détection	`logengine/analysis.py`, `logengine/detection.py`	un seul dictionnaire : le bilan
Données de test	`scripts/generate_logs.py`	un fichier JSONL de la taille demandée

Chaque étape renvoie un itérateur et ne calcule un élément que lorsque l'étape suivante le demande. C'est ce qui permet de traiter un fichier de 10 Go avec une mémoire constante (section 14).

2. Lecture en flux avec yield et yield from

La lecture fournit les lignes une par une, à la demande : le fichier reste sur disque et seule la ligne courante est en mémoire.

```
def read_lines(path: str | Path) -> Iterator[str]:
    with open(path, encoding="utf-8") as source:
        for line in source:
            yield line

def read_sources(paths: Iterable[str | Path]) -> Iterator[str]:
    for path in paths:
        yield from read_lines(path)
```

- **`yield`** fait de `read_lines` une fonction génératrice : son appel crée un générateur, et son corps ne s'exécute qu'à la première demande. `yield` fournit une ligne, suspend la fonction, puis la reprend à la demande suivante.
- **`with open(...)`** garantit la fermeture du fichier, même en cas d'erreur. Le fichier s'ouvre au premier `next()` et se ferme à la fin du parcours.
- **`yield from`** délègue à un autre générateur : `read_sources` enchaîne les fichiers dans l'ordre fourni, A en entier puis B, sans tri global. Un fichier vide ne fournit aucune ligne et la lecture passe au suivant.

Ce n'est pas la méthode `readlines()`, qui rassemble toutes les lignes dans une liste et ne conviendrait pas à un fichier de 10 Go. Un générateur épuisé ne recommence pas : pour relire, on crée un nouveau lecteur.

Les chemins ne sont pas écrits dans le code : ils sont fournis au lancement (`python main.py A.jsonl B.jsonl`). Lire d'autres fichiers demande de changer la commande, pas le lecteur.

3. Parsing et validation d'une ligne

`parse_line(line)` transforme une ligne JSON en dictionnaire typé, ou renvoie `None` si la ligne est invalide. `return None` termine seulement cet appel : la lecture continue avec la ligne suivante.

Le parsing se fait en deux temps : décoder le JSON avec `json.loads`, puis valider la structure obtenue. Du JSON valide peut en effet représenter autre chose qu'un objet, d'où le contrôle que la racine est bien un `dict`.

Le contrat des six champs, tous obligatoires :

Champ	Contrôle	Conversion
`timestamp`	chaîne, date ISO lisible	`datetime.fromisoformat()`
`level`	chaîne	aucune : `" warning "` est gardé tel quel pour le nettoyage
`ip`	chaîne	aucune
`user_id`	chaîne	aucune

Champ	Contrôle	Conversion
`connection`	liste de 3 éléments : str, int, str	`tuple()`, immuable
`payload`	`dict` ou `str`	aucune

Points de vigilance du code :

- `"timestamp" in record` teste la présence d'une clé ; présence et validité sont deux contrôles distincts. Un champ manquant n'est jamais inventé.
- Le port est contrôlé avec `type(connection[1]) is int` et non `isinstance` : `bool` hérite de `int` en Python, et `True` serait sinon accepté comme port. La chaîne `"443"` est aussi refusée, sans conversion automatique.
- `isinstance(payload, dict) or isinstance(payload, str)` accepte les deux formes de payload ; le nombre `42` est rejeté.
- Les champs supplémentaires sont conservés, et aucune règle métier n'est inventée sur les IP, les ports ou les protocoles.

Gestion des erreurs :

Situation	Réaction
Ligne vide	ignorée sans avertissement
JSON malformé, champ absent, type incorrect, date invalide	avertissement avec la cause et un extrait, `None`, puis ligne suivante
Fichier vide	cas normal, zéro événement
Fichier introuvable, accès refusé, encodage invalide	message avec le chemin, arrêt de la commande

`try`/`except json.JSONDecodeError` et `except ValueError` attrapent précisément les erreurs attendues, sans masquer les autres. Le parseur est appelé une seule fois par ligne, par `build_pipeline` (section 5) : `main.py` transmet les lignes sans les parser lui-même.

4. Génération des données de test et mode --details

Deux sortes de données servent aux essais : de petits fichiers dont on connaît le résultat à l'avance, et un gros fichier généré pour vérifier la mémoire.

Le générateur (`scripts/generate_logs.py`) produit, sérialise et écrit un événement à la fois, sans jamais construire la liste des lignes :

```
python scripts/generate_logs.py --output data/generated/logs_10go.jsonl --size-gb 10 --seed 0
```

- `generate_records` est lui-même un générateur infini (`while True` + `yield`) ; l'écriture s'arrête dès que la taille visée est atteinte.
- Il est reproductible : la même graine (`--seed`) donne exactement le même fichier.

- La dernière ligne JSON est toujours terminée : le fichier de 10 Go fait 10 000 000 021 octets, soit 21 octets de plus que la cible.
- Il refuse d'écraser un fichier existant (ouverture en mode `"xb"`).
- Le fichier généré reste dans `data/generated/`, exclu de Git : on partage le script et la commande, pas les 10 Go.

La génération de 10 Go a pris 350 s lors d'un premier essai et 1 017 s lors de l'essai complet de la section 14 ; la durée dépend de la machine.

Le mode `--details`, pour les petits fichiers, affiche le résultat de chaque ligne puis un résumé. Il s'appuie sur `parse_line_details`, qui renvoie l'événement et la cause d'un rejet, et sur `enumerate(..., start=1)` pour numéroter les lignes. Exemple avec `data/test_p1.jsonl` :

```
Ligne 1 : événement valide
Ligne 2 : ligne vide ignorée
Ligne 3 : rejetée — JSON malformé (Expecting value)
Ligne 4 : rejetée — champ manquant : timestamp
Ligne 5 : événement valide

Lecture terminée
Lignes lues : 5
Événements valides : 2
Lignes rejetées : 2
Lignes vides ignorées : 1
```

La numérotation continue d'un fichier à l'autre, et seuls trois compteurs sont gardés, jamais les événements. Cet affichage ligne par ligne est réservé aux petits fichiers : sur 10 Go, il ralentirait le traitement.

Tests de la lecture et du parsing (21 tests) : ordre de lecture, fichiers vides, fermeture des fichiers, lecture paresseuse, plusieurs sources, champs manquants, types incorrects, dates invalides, reprise après un rejet, absence de double parsing et tailles UTF-8 du générateur.

5. Nettoyage et normalisation avec filter, map et lambda

La fonction `build_pipeline` enchaîne quatre étapes paresseuses, qui ne lisent une ligne que lorsque la suite la demande :

```
nonempty_lines = filter(lambda line: line.strip() != "", lines)
parsed_records = map(parser, nonempty_lines)
valid_records = filter(None, parsed_records)
return normalize_records(valid_records)
```

- `filter` avec une `lambda` écarte les lignes vides ou faites d'espaces. La `lambda` est une petite fonction anonyme écrite directement en argument.
- `map(parser, ...)` applique le parseur à chaque ligne. La fonction est passée sans parenthèses : c'est `map` qui l'appellera, ligne par ligne.
- `filter(None, ...)` retire les `None`, que le parseur renvoie pour une ligne invalide (JSON cassé, champ manquant, mauvais type).

- `normalize_records` applique `map(normalize_record, ...)`, puis un dernier `filter` qui retire les niveaux inconnus.

`map` et `filter` ne calculent rien à leur création. Un test le vérifie : juste après l'appel à `build_pipeline`, aucune ligne n'a encore été lue.

`normalize_record` travaille sur une copie du dictionnaire reçu :

- `strip()` retire les espaces autour des chaînes et `upper()` passe le niveau et le protocole en majuscules : `" warning "` devient `"WARNING"` ;
- le tuple de connexion est déballé puis reconstruit, puisqu'un tuple ne se modifie pas ;
- une f-string construit un message lisible : `2026-11-05T10:00:01 [WARNING] user=bob ip=192.168.1.20 ...`.

Le filtre des niveaux vient après la normalisation : `" debug "` doit d'abord devenir `"DEBUG"` pour être comparé aux quatre niveaux du `frozenset` `LEVELS`.

6. Analyse de structure avec match-case

`route_value` reconnaît le format d'une valeur sans enchaîner les `if`/`elif`. Elle couvre les trois formats demandés par l'énoncé : chaîne brute, dictionnaire d'événement et tuple de connexion. Version simplifiée (chaque cas renvoie aussi la valeur d'origine) :

```
match value:
    case str():
        return {"kind": "text", "summary": value}
    case {"event": str() as event, "status": str() as status}:
        return {"kind": "event", "summary": f"{event} : {status}"}
    case {"level": str() as level, "code": int() as code}:
        return {"kind": "event", "summary": f"{level} code {code}"}
    case dict():
        return {"kind": "event", "summary": "événement structuré"}
    case (str() as ip, int() as port, str() as protocol) if isinstance(
        value, tuple
    ):
        return {"kind": "connection", "summary": f"{protocol} {ip}:{port}"}
    case _:
        return {"kind": "unknown", "summary": "format non reconnu"}
```

Trois sortes de motifs sont utilisées :

- **motif de classe** (`str()`, `dict()`) : vérifie seulement le type ;
- **motif de dictionnaire** (`{"level": str() as level, "code": int() as code}`) : vérifie que les clés existent et que leurs valeurs ont le bon type, puis les range dans des variables. Les autres clés sont acceptées. Un code écrit `"403"` (texte) ne correspond pas et retombe sur le cas `dict()` ;
- **motif de séquence** (`(str() as ip, int() as port, str() as protocol)`) : déballe les trois éléments du tuple. Il accepterait aussi une liste ; la condition `if isinstance(value, tuple)` impose un vrai tuple.

L'ordre des cas compte : les motifs précis passent avant `dict()`, qui attraperait sinon tous les dictionnaires. Le cas `_` joue le rôle d'un `else`.

Chaque événement normalisé reçoit `payload_description` et `connection_description`, qui indiquent le format reconnu et son résumé, par exemple `login : failure` ou `HTTPS 192.168.1.20:443`.

7. Moteur de règles avec *args et **kwargs

Une règle est une simple fonction : elle reçoit un événement et la configuration, et renvoie une alerte (un dictionnaire) ou `None`. Deux règles sont fournies :

- `rule_blacklist` : l'IP de l'événement figure dans la blacklist ;
- `rule_critical` : l'événement est de niveau CRITICAL.

Le moteur a cette signature :

```
def apply_security_rules(*rules, records, **config):
```

- `*rules` regroupe dans un tuple toutes les fonctions passées en arguments positionnels : zéro, une ou plusieurs.
- `records` est placé après `*rules` : il doit donc être donné par son nom (`records=flux`), sinon Python le prendrait pour une règle.
- `**config` regroupe les paramètres nommés restants (`blacklist=...`) dans un dictionnaire, transmis tel quel à chaque règle avec `rule(record, **config)`.

Les fonctions sont manipulées comme des objets : rangées dans un tuple, parcourues avec `for`, puis appelées. Ajouter une règle ne demande aucune modification du moteur :

```
apply_security_rules(rule_blacklist, records=flux, blacklist=ips)
apply_security_rules(rule_blacklist, rule_critical, records=flux, blacklist=ips)
```

Le moteur renvoie chaque événement avec `yield`, même sans alerte, car l'analyse en a besoin pour ses statistiques. La liste `alerts` ne concerne que l'événement en cours : aucune liste globale d'alertes n'est construite. Une alerte a toujours la forme `type`, `timestamp`, `ip`, `user_id`, `message`, la même que celles du détecteur de force brute (section 10).

Typage. Les fonctions sont annotées pour documenter leurs entrées et sorties, comme le demande le critère de qualité :

```
def build_pipeline(
    lines: Iterable[str],
    parser: Callable[[str], dict | None] = parse_line,
) -> Iterator[dict]:

def apply_security_rules(
    *rules: Callable[..., dict | None],
    records: Iterable[dict],
    **config: object,
) -> Iterator[dict]:
```

- `Iterable[str]` : un objet parcourable avec `for`, ici des lignes ;

- `Iterator[dict]` : un objet qui fournit ses dictionnaires un par un ;
- `Callable[[str], dict | None]` : une fonction qui reçoit une chaîne et renvoie un dictionnaire ou `None`, comme le parseur ;
- `Callable[..., dict | None]` : une règle, quels que soient ses arguments.

Ces annotations ne vérifient rien pendant l'exécution : elles servent à la lecture et aux outils de vérification.

8. Audit des portées avec globals() et locals()

`export_audit(path, global_scope, local_scope)` écrit un fichier JSON qui associe chaque nom de variable au nom de son type :

```
{"globals": {"PROJECT_DIR": "WindowsPath", ...},
 "locals": {"blacklist": "frozenset", "summary": "dict", ...}}
```

- `globals()` renvoie le dictionnaire des variables du module, `locals()` celui de la fonction en cours. Ils sont appelés dans `main.py` puis passés à l'exporteur. Appelés dans `audit.py`, ils décriraient les variables de `audit.py` et non celles du programme principal.
- Seuls les noms et les types sont exportés, pas les valeurs : elles peuvent être sensibles, et certaines (modules, générateurs) ne s'écrivent pas en JSON.
- `type(value).__name__` ne parcourt pas un générateur : l'audit ne consomme pas le flux de logs.

9. Rôle du module d'analyse

Le module d'analyse transforme le flux d'événements nettoyés en un seul résultat : le bilan, qui compte les événements, compare les IP à la blacklist et rassemble les alertes. C'est le dernier maillon avant l'affichage par `main.py`.

Ce qu'il reçoit. Un flux d'événements, un par un, déjà lus par le module de lecture, puis nettoyés et enrichis par le pipeline et le moteur de règles. Chaque événement est un dictionnaire qui respecte le contrat commun : `timestamp` (datetime), `level`, `ip`, `user_id`, `connection` (tuple IP, port, protocole), `payload` (dictionnaire ou texte) et `alerts` (liste d'alertes ajoutées par le moteur de règles).

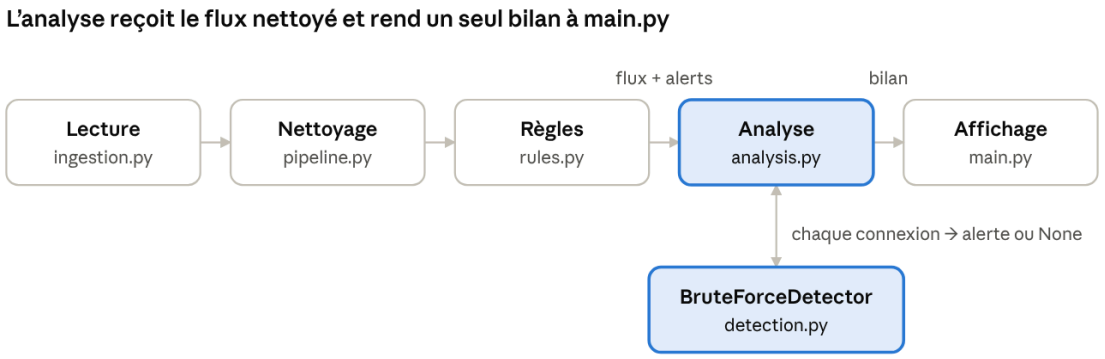
Ce qu'il produit. Un dictionnaire unique qui répond à trois questions :

- Qu'est-ce qui s'est passé ? Le total d'événements, le nombre par niveau et par utilisateur.
- Quelles IP posent problème ? Les IP actives, celles en erreur, et leur comparaison avec la blacklist.
- Quelqu'un tente-t-il de forcer un mot de passe ? Les alertes de force brute, avec celles du moteur de règles.

Comment il est appelé. `main.py` appelle `analyze(checked, blacklist=blacklist, threshold=threshold)`. Le flux n'est parcouru qu'une seule fois et aucun événement n'est conservé : seules des notes (compteurs, ensembles, état du détecteur) restent en mémoire.

10. Architecture du module d'analyse

Le module tient en deux fichiers : `analysis.py` construit le bilan, `detection.py` détecte la force brute. `analyze` appelle le détecteur dans la même boucle, pour ne parcourir le flux qu'une fois.



chaîne de traitement · module d'analyse en couleur

Les événements traversent la chaîne un par un ; `analyze` envoie chaque tentative de connexion au détecteur, puis rend le bilan à `main.py`.

Fichier	Fonction ou classe	Entrée → sortie
`logengine/analysis.py`	`analyze(records, blacklist, threshold)`	flux d'événements → bilan (dict)
`logengine/analysis.py`	`compare_with_blacklist(active_ips, blacklist)`	deux ensembles d'IP → 5 ensembles comparés
`logengine/detection.py`	`is_login_attempt(record)`	un événement → `True` si c'est une connexion
`logengine/detection.py`	`BruteForceDetector(threshold).process(record)`	un événement → une alerte ou `None`
`tests/sample_records.py`	`RECORDS`, `BLACKLIST`	les 10 événements du jeu de référence, tels qu'ils sortent du moteur de règles
`tests/test_analysis.py`, `tests/test_detection.py`	22 tests	vérifient le bilan et les règles de détection

Une alerte a toujours la même forme, qu'elle vienne du moteur de règles ou du détecteur : `type`, `timestamp` (texte ISO), `ip`, `user_id`, `message`. `analyze` peut donc les compter ensemble.

11. Notions Python de l'analyse

Chaque notion imposée par l'énoncé pour l'agrégation et la détection a un rôle précis dans le code, pas seulement une présence.

Notion	Où	Pourquoi
`defaultdict(int)`	`analyze` : `by_level`, `by_user`, `alerts_by_type` ; détecteur : séries d'échecs	Une clé jamais vue commence à 0 : `+= 1` fonctionne sans `KeyError` ni test préalable.
`set`	`active_ips`, `error_ips`	Une IP n'est gardée qu'une fois, quel que soit son nombre d'apparitions.
`frozenset`	`BLACKLIST`, `ERROR_LEVELS`	Ensembles de référence immuables : ils ne peuvent pas être modifiés par erreur pendant le traitement.
`|` `&` `-` `^`	`compare_with_blacklist`	Union, intersection, différences dans les deux sens et différence symétrique entre IP actives et blacklist.
Tuple	clé `(ip, user_id)` du détecteur	Un tuple est immuable, donc utilisable comme clé de dictionnaire.
`while` + `next()` + `StopIteration`	boucle principale d'`analyze`	Lit le flux à la demande, sans le transformer en liste ; l'exception marque la fin du flux.
`break`	`analyze` et détecteur	Sort de la lecture à la fin du flux ; arrête la recherche du statut dès qu'il est trouvé.
`continue`	`analyze` et détecteur	Passe les événements qui ne sont pas des connexions ; passe au statut suivant quand il ne correspond pas.
`else` de boucle	`process` du détecteur	S'exécute seulement si la boucle `for` se termine sans `break`, donc quand le statut n'est ni `success` ni `failure`.
Classe	`BruteForceDetector`	Garde l'état des séries entre deux appels, et donc d'un fichier à l'autre dans un même lancement.
Typage, exceptions, `logging`	tout le module	Signatures typées ; `ValueError` si le seuil est inférieur à 1 ; avertissement pour un statut inconnu, sans arrêter le traitement.

Le point le plus délicat est le `else` de boucle. Il ne joue pas le rôle du `else` d'un `if` : il veut dire « la boucle a tout parcouru sans trouver ». Ici, il signale un statut de connexion inconnu, que le détecteur ignore sans modifier la série.

12. Détection de la force brute

Le détecteur émet une seule alerte quand un couple (IP, utilisateur) atteint le seuil d'échecs de connexion consécutifs, 3 pour la démonstration. Il applique les règles fixées par le guide d'équipe :

- l'état est suivi séparément pour chaque couple `(ip, user_id)` ;
- seul un payload `{"event": "login"}` compte : une requête en échec ou un message texte ne touche pas la série ;
- un échec du couple allonge sa série, une réussite la remet à zéro ;
- l'alerte part quand la série vaut exactement le seuil (`==` et non `>=`), donc une seule fois par série ;
- l'état est conservé d'un fichier à l'autre dans un même lancement.

Sur le jeu de référence, la série de bob depuis l'IP 192.168.1.20 donne exactement une alerte :

Ligne	Événement de bob	Série après l'événement	Alerte
2	échec de connexion	1	non
4	échec de connexion	2	non
5	échec de connexion	3	**oui, force brute**
7	connexion réussie	0 (remise à zéro)	non
8	échec de connexion	1	non

L'échec de la ligne 10 (alice, requête `/report`) n'est pas une tentative de connexion : il est ignoré par le détecteur.

13. Démonstration sur le jeu de référence

Lancée de bout en bout avec `main.py` sur `data/demo.jsonl`, la chaîne complète affiche un bilan identique aux résultats attendus par le guide : 10 événements valides, une alerte blacklist et une alerte de force brute.

```
python main.py data/demo.jsonl --blacklist data/blacklist.txt --threshold 3
```

La ligne JSON incomplète du fichier est rejetée avec un avertissement, et la ligne vide est ignorée, avant d'atteindre l'analyse.

Élément du bilan	Résultat
`total`	10
`by_level`	INFO 4 ; WARNING 3 ; ERROR 2 ; CRITICAL 1
`by_user`	bob 5 ; alice 3 ; carol 1 ; service 1
`active_ips`	.10, .20, .30, .66
`error_ips`	.10, .20, .66
`ip_sets.intersection`	.66 (IP blacklistée mais active)
`ip_sets.active_only`	.10, .20, .30

Élément du bilan	Résultat
`ip_sets.blacklist_only`	.99
`ip_sets.union`	.10, .20, .30, .66, .99
`ip_sets.symmetric_difference`	.10, .20, .30, .99
`alerts_by_type`	blacklist 1 ; brute_force 1
`alert_samples`	force brute pour bob/.20 à 10:00:04 ; blacklist pour carol/.66 à 10:00:05

Toutes les IP sont dans le réseau 192.168.1.

Essais du nettoyage et des règles. La démonstration du moteur de règles (`python main.py --demo-p2`) porte sur trois dictionnaires :

Scénario	Événements	Alertes
`rule_blacklist` seule	3	1 (blacklist, carol)
`rule_blacklist` et `rule_critical`	3	2 (blacklist et critical, carol)

Le fichier de test de la lecture (`data/logs_test_p1.jsonl`), passé dans le pipeline, donne 10 événements conservés (INFO 4, WARNING 3, ERROR 2, CRITICAL 1), toutes les connexions reconnues comme tuples, un seul payload texte (david), aucune alerte blacklist et une alerte CRITICAL (admin). `rule_critical` est désactivée dans la chaîne complète pour conserver les résultats de référence.

14. Vérifications et mémoire

Le projet est couvert par 75 tests `unittest`, qui passent tous, dont 22 pour l'analyse et la détection. Sur un fichier de 10 Go, la chaîne complète garde une mémoire inférieure à 20 Mo.

Commande : `python -m unittest discover -s tests -t .`

Fichier	Tests	Ce qui est vérifié
`test_analysis.py`	10	total, niveaux, utilisateurs, IP actives et en erreur, 5 ensembles de la blacklist, alertes par type et exemples, plafond de 20 exemples avec comptage complet (25 alertes), flux vide
`test_detection.py`	12	seuil non atteint puis atteint, format de l'alerte, une seule alerte par série, remise à zéro, nouvelle alerte après une réussite, activité intercalée, deux utilisateurs derrière une même IP, requête et message texte ignorés, statut inconnu signalé, série répartie sur deux fichiers, seuil invalide refusé

Les tests du nettoyage, des règles et de l'audit couvrent notamment :

- la normalisation, sans modification du dictionnaire d'origine ;

- les trois formats de `match-case` et le cas non reconnu ;
- la poursuite du traitement après une ligne vide, invalide ou un niveau inconnu ;
- le traitement à la demande du pipeline ;
- le moteur avec zéro, une ou deux règles, et la conservation des événements sans alerte ;
- l'audit, qui n'exporte pas les valeurs et ne consomme pas de générateur.

Les tests passent le jeu de référence sous la forme `iter(RECORDS)` : un itérateur ne se lit qu'une fois, comme le vrai flux. Un code qui le parcourrait deux fois échouerait.

Ce qui reste en mémoire. Aucun événement n'est conservé. `analyze` ne garde que des compteurs, deux ensembles d'IP, l'état du détecteur et au plus 20 exemples d'alertes. Leur taille dépend du nombre d'IP et d'utilisateurs distincts, pas de la taille du fichier. Le détecteur retire un couple dès qu'il réussit à se connecter.

Mesures sur gros volume. Fichiers produits par `scripts/generate_logs.py`, puis traités par `main.py` (lecture, nettoyage, règle blacklist, analyse, seuil de 3) :

```
python scripts/generate_logs.py --output data/generated/logs_10go.jsonl --size-gb 10
--seed 0
python main.py data/generated/logs_10go.jsonl --blacklist data/blacklist.txt
--threshold 3
```

Fichier généré	Événements	Durée de la chaîne	Pic mémoire	Environnement
10 Mo	55 993	0,5 s	16 Mo	Linux
500 Mo	2 799 227	23,5 s	16 Mo	Linux
10 Go	55 983 446	1 164 s (environ 48 000 événements/s)	19,5 Mo	Windows, Python 3.14

Le volume est multiplié par 1 000 entre le premier et le dernier fichier, et la mémoire reste sous 20 Mo : ni les événements ni les alertes ne sont accumulés. Pour 10 Go, la génération a duré 1 017 s, aucune ligne n'a été rejetée, et la mémoire a été relevée sur le processus Python lui-même (working set Windows), sans instrumentation qui ralentirait le parcours.

Le générateur tire les données au hasard sur 32 utilisateurs et les IP `192.168.1.1` à `.32` : aucune n'est dans la blacklist de test (`.66` et `.99`), d'où 0 alerte blacklist. Les échecs de connexion étant tirés au hasard, les alertes de force brute sont très nombreuses (2 623 123 sur 10 Go, dont 20 gardées en exemple) : ces chiffres testent la tenue en charge, pas un scénario réaliste.

15. Limites et améliorations possibles

La détection suit volontairement des règles simples, fixées par l'équipe, et ne repère donc pas toutes les attaques.

- **Pas de fenêtre de temps.** Trois échecs espacés de plusieurs jours déclenchent la même alerte que trois échecs en trois secondes. Amélioration : ne compter que les échecs des N dernières minutes.

- **Suivi par couple (IP, utilisateur).** Une IP qui essaie un mot de passe sur beaucoup de comptes différents, ou une attaque répartie sur plusieurs IP, n'est pas détectée. Amélioration : un second compteur par IP seule.
- **Statuts inconnus ignorés.** Un statut autre que `success` ou `failure` est signalé dans le journal mais ne compte ni comme échec ni comme réussite.
- **Exemples d'alertes limités.** Seules les 20 premières alertes sont gardées comme exemples, toutes étant comptées. Sur un très gros fichier, les plus récentes n'apparaissent donc pas.
- **Mémoire liée aux identités.** Les ensembles d'IP grandissent avec le nombre d'IP distinctes. Avec des millions d'adresses différentes, ils finiraient par peser.

Côté nettoyage, règles et audit :

- **IP non validées.** Le pipeline nettoie seulement les espaces autour des IP et ne vérifie pas leur syntaxe ; le contrat commun ne le demande pas.
- **Copie superficielle.** `record.copy()` ne copie pas le payload : un dictionnaire de payload reste partagé avec l'original. Les fonctions de nettoyage ne le modifient pas.
- **`rule_critical` désactivée** dans la chaîne complète, pour conserver les résultats de référence.
- **Audit ponctuel.** L'audit décrit seulement la portée choisie par l'appelant, à l'instant de l'appel.

Côté lecture et parsing :

- **Une ligne géante reste une limite.** La mémoire de la lecture dépend surtout de la plus longue ligne du fichier.
- **Rejets sans numéro de ligne.** Le parseur signale la cause et un extrait de la ligne rejetée, mais pas son numéro, car son interface ne reçoit que la ligne ; seul le mode `--details` numérote.
- **Validation de type seulement.** Les types sont contrôlés, mais pas la validité métier des IP, des ports ou des protocoles.
- **Mesure mémoire.** `tracemalloc` (option `--measure`) ne suit que les allocations Python et ralentit le parcours ; l'essai de 10 Go a donc été mesuré sur le processus lui-même. Les durées dépendent de la machine.

Conclusion

Le moteur répond à l'objectif fixé : il lit, nettoie, contrôle et analyse des logs volumineux en un seul passage, et traite un fichier de 10 Go avec une mémoire constante, sous 20 Mo.

Ce qui a été réalisé. Chaque partie de l'énoncé est couverte par du code testé :

Partie de l'énoncé	Réalisation	Preuve
Ingestion et générateurs	lecture avec `yield` et `yield from`, parsing en dictionnaires typés, tuples de connexion	tests de lecture et de parsing, mode `--details`
Pipeline fonctionnel	`filter`, `map`, `lambda`, normalisation, `match-case` sur trois formats	tests du pipeline, traitement à la demande vérifié

Partie de l'énoncé	Réalisation	Preuve
Agrégation et détection	`set`, `frozenset` et leurs opérateurs, `defaultdict`, détecteur avec `while`, `for`, `break`, `continue`, `else`	bilan identique aux résultats de référence
Moteur de règles et introspection	`apply_security_rules(*rules, records, **config)`, audit avec `globals()` et `locals()`	démonstration avec une et deux règles
Environnement et qualité	venv, package `logengine`, `main.py`, typage, exceptions, PEP 8	75 tests `unittest` réussis

Ce que le projet nous a appris.

- **Penser en flux.** Enchaîner des itérateurs plutôt que des listes change toute la conception : chaque étape doit pouvoir travailler sur un seul événement, sans revenir en arrière.
- **Se mettre d'accord avant de coder.** Le contrat commun (six champs, format des alertes, forme du bilan) a permis de développer les modules en parallèle, chacun avec ses propres données de test, puis de les assembler sans réécriture.
- **Prouver par les tests.** Fixer les résultats attendus du jeu de référence avant d'écrire le code a servi de repère à chaque étape, et a rendu l'intégration finale vérifiable en une commande.

Perspectives. Les limites listées en section 15 tracent les prochaines étapes : une fenêtre de temps et un compteur par IP pour la détection, une validation métier des IP et des ports, et un générateur plus réaliste, qui inclurait des IP blacklistées, des lignes invalides et des attaques placées volontairement. Un notebook de visualisation du bilan, autorisé par l'énoncé pour la démonstration, compléterait la présentation des résultats.